

HCL AppScan Report

Report Type: Security Report

Scan Name: DAST 2026-09-10 <https://aaca.ntua.edu.tw/>

Technology: DAST

Report created at: Friday, September 11, 2026

This report was generated by HCL AppScan v2.1

Summary of security issues

Medium severity issues: 1

Informational severity issues: 1

Total security issues: 2

Scan Information

Scan started: Thursday, September 10, 2026 9:00:29 AM (UTC)

Test policy: Default

Test Optimization: No Optimization

Hosts

Host: aaca.ntua.edu.tw

Port: 443

Operating system: Unknown

Web server: Unknown

Application Server: Any

Login Settings

Login method: None

Table of Contents

Summary

- [Issues](#)

Issue Types

- [ADNS Blind SSRF on Sensitive Headers](#)
- [Client-Side \(JavaScript\) Cookie References](#)

How to Fix

- [ADNS Blind SSRF on Sensitive Headers](#)
- [Client-Side \(JavaScript\) Cookie References](#)

Summary

Total security issues: **2**

Issue Types: **2**

		Number of Issues
ADNS Blind SSRF on Sensitive Headers	1	
Client-Side (JavaScript) Cookie References	1	

Critical High Medium Low Informational

Issues - By Issue Types:

M DAST: ADNS Blind SSRF on Sensitive Headers 1

How to Fix: [ADNS Blind SSRF on Sensitive Headers](#)

Issue 1 of 1

Issue ID:	2c48443e-f36b-1410-89cd-00f37f576f24
Severity:	Medium
Status	Open
Location	https://aaca.ntua.edu.tw/service/banner/querylist
Domain	aaca.ntua.edu.tw
Element	QueryList (Page)
Path	/service/banner/querylist
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	6.0
Date Created	Thursday, September 10, 2026
Last Updated	Thursday, September 10, 2026
CWE:	918

Issue 1 of 1 - Details

Difference: Header `Host` manipulated from: `aaca.ntua.edu.tw` to:

`v3-ping-8685-6890f707-27fe-47c8-a5d2-5a2959283746.adns.appsechcl.com`

Reasoning: The AppScan external DNS server received a DNS query containing the Variant ID of this test, and therefore concluded the application is vulnerable.

Test Requests and Responses:

```
POST /Service/Banner/QueryList HTTP/1.1
Host: v3-ping-8685-6890f707-27fe-47c8-a5d2-5a2959283746.adns.appsechcl.com
Connection: keep-alive
sec-ch-ua-platform: "Linux"
X-XSRF-TOKEN: CfdJ8AA3XS411DRKt-gAtKDiBdzsoTQCeLAW4d9xvOYaSdk-DNuH-Y4soERXg6Odc0r0IluaBOZe8h-SW0yZM009vyGJ_-vo6w-bIPWVN8umd7aOCiGI6bhv38AMfPaO6_CV8hfaPwROO2IfN_fqHVJK2Ek
Accept-Language: en-US
sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/145.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/json
Origin: https://aaca.ntua.edu.tw
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://aaca.ntua.edu.tw/en/Sitemap
Accept-Encoding: gzip
Cookie: wcms.lang=en; __Host-WCMS-Antiforgery=CfdJ8AA3XS411DRKt-gAtKDiBdzGuaiySHn5xOsSzzfN0ONGQ5ISOcLTANaONWwAk72xOCzw9PaypqWwasiQ9iBKoq8TpTKgjl9CefwUyZ_cDPFYgnxj0L1TsqafoqbqfspMilKnQWFFkjnUynNPfzIRBZg; XSRF-TOKEN=CfdJ8AA3XS411DRKt-gAtKDiBdzsoTQCeLAW4d9xvOYaSdk-DNuH-Y4soERXg6Odc0r0IluaBOZe8h-SW0yZM009vyGJ_-vo6w-bIPWVN8umd7aOCiGI6bhv38AMfPaO6_CV8hfaPwROO2IfN_fqHVJK2Ek; wcms.visitor=bd0344592d7a4d4e9445c9eae2e859ad
Content-Length: 28

{
  "Fields": [
  ],
  "Condition": ""
}

HTTP/1.1 403 Forbidden
Cache-Control: no-store, no-cache, must-revalidate, proxy-revalidate
Pragma: no-cache
Content-Type: text/html
Expires: 0
Vary: Origin, Accept-Encoding
access-control-allow-credentials: true
access-control-allow-origin: https://aaca.ntua.edu.tw
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
```

```

referrer-policy: strict-origin-when-cross-origin
permissions-policy: geolocation=(), microphone=(), camera=(), fullscreen=(self)
cross-origin-opener-policy: same-origin
cross-origin-embedder-policy: require-corp
cross-origin-resource-policy: same-origin
x-permitted-cross-domain-policies: none
content-security-policy: default-src 'none'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'; form-action 'none'
strict-transport-security: max-age=16070400; includeSubDomains
X-Powered-By:
Date: Thu, 10 Sep 2026 10:20:17 GMT
Content-Length: 1215

/**
AppScan message: The response body is omitted as it is not needed to detect the vulnerability.
**/

```

[Go to Table of Contents](#)

I DAST: Client-Side (JavaScript) Cookie References 1

How to Fix: [Client-Side \(JavaScript\) Cookie References](#)

Issue 1 of 1

Issue ID:	3a47443e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/assets/index-cvw6f4ou.js
Domain	aaca.ntua.edu.tw
Element	const __vite_mapDeps=(i,m=__vite_mapDeps,d=(m.f (m.f=["assets/LoadSpecCss-DvpspM99.css","assets/L... (Page)
Path	/assets/index-cvw6f4ou.js
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, September 10, 2026
Last Updated	Thursday, September 10, 2026
CWE:	602

Issue 1 of 1 - Details

Reasoning: AppScan found a reference to cookies in the JavaScript.

Test Requests and Responses:

```

GET /assets/index-CVW6f4Ou.js HTTP/1.1
Host: aaca.ntua.edu.tw
Connection: keep-alive
Origin: https://aaca.ntua.edu.tw
sec-ch-ua-platform: "Linux"
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/145.0.0.0 Safari/537.36
sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
sec-ch-ua-mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: script
Referer: https://aaca.ntua.edu.tw/Server
Accept-Encoding: gzip
Accept-Language: en-US
Content-Length: 0

HTTP/1.1 200 OK
Cache-Control: public, max-age=31536000, immutable
Keep-Alive: timeout=5
Accept-Ranges: bytes
ETag: W/"17d4ab-1a085a42410"
Vary: Accept-Encoding
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=(), fullscreen=(self)
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Cross-Origin-Resource-Policy: same-origin
X-Permitted-Cross-Domain-Policies: none
Strict-Transport-Security: max-age=16070400; includeSubDomains
Content-Security-Policy: default-src 'none'; require-trusted-types-for 'script'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'
X-Powered-By:
Date: Thu, 10 Sep 2026 10:27:31 GMT
Content-Type: text/javascript; charset=utf-8
Last-Modified: Wed, 09 Sep 2026 10:08:42 GMT
Content-Length: 1561771

```

```

const vite_mapDeps=(i,m=__vite__mapDeps,d=(m.f||(m.f=["assets/LoadSpecCss-DvpspM99.css","assets/LoadSpecCss_Server-
CvGY3VMi.css","assets/LoadFeaturesCss-DZPbrguC.js","assets/LoadFeaturesCss-D6S3uYhx.css","assets/cms_content_format-
NvYllcok.css","assets/LoadFeaturesCss_Client-DC75pXl9.js","assets/LoadFeaturesCss_Client-Cz6Vl-KF.css"])))=>i.map((i=>d[i]);
function uL(e,t){for(var n=0;n<t.length;n++){const r=t[n];if(typeof r!="string"&&Array.isArray(r)){for(const a in r){if(a!="default"&&!(a in e)){const
s=Object.getOwnPropertyDescriptor(r,a);s&&Object.defineProperty(e,a,s.get?s:{enumerable:!0,get:()=>r[a]}}}}return
Object.freeze(Object.defineProperty(e,Symbol.toStringTag,{value:"Module"}))}}function(){const
t=document.createElement("link").relList;if(t&&t.supports&&t.supports("modulepreload"))return;for(const a of
document.querySelectorAll("link[rel=modulepreload]"))(a).new MutationObserver(a=>{for(const s of a){if(s.type=="childList")for(const o of
s.addedNodes)o.tagName=="LINK"&&o.rel=="modulepreload"&&(o).observe(document,{childList:!0,subtree:!0});function n(a){const s=
{ };return a.integrity&&(s.integrity=a.integrity),a.referrerPolicy&&(s.referrerPolicy=a.referrerPolicy),a.crossOrigin=="use-credentials"?
s.credentials="include":a.crossOrigin=="anonymous"?s.credentials="omit":s.credentials="same-origin",s}function r(a){if(a.ep)return;a.ep=!0;const
s=n(a);fetch(a.href,s)}const o5="modulepreload",l5=function(e){return"/"+e},CD={},sl=function(t,n,r){let a=Promise.resolve();if(n&&n.length>0)
{let o=function(f){return Promise.all(f.map(g=>Promise.resolve(g)).then(y=>({status:"fulfilled",value:y})),y=>
({status:"rejected",reason:y})))};document.getElementsByTagName("link");const c=document.querySelector("meta[property=csp-
nonce]"),d=c?.nonce||c?.getAttribute("nonce");a=o(n.map(f=>{if(f=l5(f),f in CD)return;CD[f]=!0;const
g=f.endsWith(".css"),y=g?["rel=stylesheet"]:"";if(document.querySelector(`link[href="$ {f}"]$ {y}`))return;const
p=document.createElement("link");if(p.rel=g?"stylesheet":"o5,g]]
(p.as="script"),p.crossOrigin="",p.href=f,d&&p.setAttribute("nonce",d),document.head.appendChild(p),g)return new Promise((w,b)=>
{p.addEventListener("load",w),p.addEventListener("error",l)=>b(new Error(`Unable to preload CSS for $ {f}`))}));function s(o){const c=new
Event("vite:preloadError",{cancelable:!0});if(c.payload=o,window.dispatchEvent(c),!c.defaultPrevented)throw o}return a.then(o=>{for(const c of o||
[])c.status=="rejected"&&s(c.reason);return t().catch(s)});function Cc(e){return
e&&e.__esModule&&Object.prototype.hasOwnProperty.call(e,"default"?e.default:e}var Mv={exports:{},tm={},Ev={exports:{},kt={}};/**
 * @license React
 * react.production.min.js
 *
 * Copyright (c) Facebook, Inc. and its affiliates.
 *
 * This source code is licensed under the MIT license found in the
 * LICENSE file in the root directory of this source tree.
 */var _D;function c5(){if(_D)return kt;_D=1;var
e=Symbol.for("react.element"),t=Symbol.for("react.portal"),n=Symbol.for("react.fragment"),r=Symbol.for("react.strict_mode"),a=Symbol.for("react.profi
ler"),s=Symbol.for("react.provider"),o=Symbol.for("react.context"),c=Symbol.for("react.forward_ref"),d=Symbol.for("react.suspense"),f=Symbol.for("rea
ct.memo"),g=Symbol.for("react.lazy"),y=Symbol.iterator;function p(A){return A===null||typeof A!="object"?null:(A=y&&A[y])||A["@@iterator"],typeof
A=="function"?A:null}var w={isMounted:function(){return!1},enqueueForceUpdate:function(){},enqueueReplaceState:function()
{}},enqueueSetState:function(){},b=Objec
...
...
...

```

[Go to Table of Contents](#)

How to Fix

M ADNS Blind SSRF on Sensitive Headers

Cause

Incorrect processing, sanitation, or validation of user input that contain elements later joined with URI

Risk

It is possible to manipulate the server into processing, sending, or retrieving information in an unintended, insecure manner

The application sends requests to resources using externally-influenced input, and fails to neutralize elements that could be used to abuse internal or external functionality.

This can be used to cause the server to initiate unintended requests on behalf of the attacker.

There are three main examples of abuse of this vulnerability:

[1] An attacker could cause the server to leak important information, by sending information normally accessible only by the application to an external server.

[2] An attacker could cause the server to retrieve information from an untrusted source and process, store, or present it to the user.

[3] In an environment where services communicated internally, an attacker could cause an action to be performed by the server, using the server's privileges.

Example:

The application includes a common 3rd Party service that exposes REST APIs to manage users profiles.

...

```
$userPostData = http_build_query($formFields); # data will go into POST request $options
```

...

```
file_get_contents($fetchUserBioAPI, false, stream_context_create($options)); # this calls the API denoted by $service to perform an action
```

...

Example of an exploitation of this issue:

[Original HTML Form]

...

```
<FORM METHOD=GET ACTION="/userData">
```

```
<INPUT TYPE=HIDDEN NAME="fetchUserBioAPI" VALUE="https://companydomain.com/user/profile/bio">
```

...

```
</FORM>
```

[Manipulated HTML Form]

...

```
<FORM METHOD=GET ACTION="/userData">
```

```
<INPUT TYPE=HIDDEN NAME="fetchUserBioAPI" VALUE="https://localhost/activeSessions">
```

...

</FORM>

The original intent of the form was to show the user biography by calling "fetchUserBioAPI" service. By manipulating the request to call "https://localhost/activeSessions" the server will retrieve the active sessions of all the users who are logged in, an action which should have been reserved only to an administrator.

Since the call is coming from the application server and not directly to the user management service, the request is honored.

Sample Exploit: Atlassian Confluence SSRF / Remote Code Execution

[object Object]

Owasp Server Side Request Forgery:

[object Object]

Fix recommendation

There are several mitigation techniques:

[1] If possible, use library calls rather building the requests yourself.

[2] Strategy: Whitelist

Create a whitelist of servers and services that are permitted, preferably per role or function.

[3] Strategy: Libraries or Frameworks

Use a vetted library or framework that prevents this weakness occurring, or at least provides constructs that make it easier to avoid.

For example, consider using the ESAPI Encoding control

<https://owasp.org/www-project-enterprise-security-api>

or a similar tool, library, or framework. These can help to encode output.

[4] Strategy: Input Validation

Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a whitelist of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a blacklist). However, blacklists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

When constructing OS command strings, use stringent whitelists that limit the character set based on the expected value of the parameter in the request. This will indirectly limit the scope of an attack, but this technique is less important than proper output encoding and escaping.

Note that proper output encoding, escaping, and quoting is the most effective solution for preventing OS command injection, although input validation may provide some defense-in-depth. This is because it effectively limits what will appear in output. Input validation will not always prevent OS command injection, especially if you are required to support free-form text fields that could contain arbitrary characters. For example, when invoking a mail program, you might need to allow the subject field to contain otherwise-dangerous inputs like ";" and ">" characters, which would need to be escaped or otherwise handled. In this case, stripping the character might reduce the risk of OS command injection, but it would produce incorrect behavior because the subject field would not be recorded as the user intended. This might seem to be a minor inconvenience, but it could be more important when the program relies on well-structured subject lines in order to pass messages to other components.

Even if you make a mistake in your validation (such as forgetting one out of 100 input fields), appropriate encoding is still likely to protect you from injection-based attacks. As long as it is not done in isolation, input validation is still a useful technique, since it may significantly reduce your attack surface, allow you to detect some attacks, and provide other security benefits that proper encoding does not address.

[5] Strategy: Environment Hardening

Run your code using the lowest privileges that are required to accomplish the necessary tasks

<https://buildsecurityin.us-cert.gov/daisy/bsi/articles/knowledge/principles/351.html>

If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations.

CWE

918

Cause

Cookies are created at the client side

Risk

The worst case scenario for this attack depends on the context and role of the cookies that are created at the client side

A cookie is a piece of information usually created by the Web server and stored in the Web browser.

The cookie contains information used by web applications mainly (but not only) to identify users and maintain their state.

AppScan detected that the JavaScript code at the client side is used to manipulate (either create or modify) the site's cookies.

It is possible for an attacker to view this code, understand its logic, and use it to compose his own cookies, or modify existing ones, based on this knowledge.

The damage an attacker may cause depends on how the application uses its cookies, or what information it stores in them.

Among other things, cookie manipulation may lead to session hijacking or privilege escalation.

Other vulnerabilities caused by cookie poisoning contain SQL injection and Cross-Site scripting.

Fix recommendation

[1] Avoid placing business/security logic at the client side.

[2] Find and remove insecure client-side Javascript code which may pose a security threat to the site.

CWE

602

External references

- [WASC Threat Classification: Information Leakage](#)

[Go to Table of Contents](#)